Husky Park Predictor: AI-Driven Parking Spot Probability Dashboard

Phase 3: Web Interface Design

CSCI 414/514 - Platform Based Development

Section: Group 8

Group Members: Chidubem Okoye, Yvonne Onmakpo, Tonika Devi Avanigadda

A. Functional Design

Overview

The web interface for HuskyParkPredictor is designed to provide users with real-time and predictive parking availability information through an intuitive and efficient interface. The system supports full CRUD operations, search and filtering, and data visualization while integrating seamlessly with both relational and NoSQL databases.

Core Functionalities

1. Create (Add Operations)

Users can submit parking availability reports, create accounts, and (if authorized) add permits or campus events.

Data Involved:

- CrowdsourcedReport
- User
- Permit
- CampusEvent

Backend Integration:

The frontend sends POST requests to REST API endpoints such as /api/reports and /api/users. Structured data is stored in PostgreSQL, while enriched or unstructured data (e.g., metadata, optional media) is stored in MongoDB.

2. Update Operations

Users can update their profiles and permits. Administrators can modify parking lots, events, and prediction parameters.

Data Involved:

- User
- Permit
- ParkingLot
- CampusEvent

Backend Integration:

PUT/PATCH requests are sent to endpoints such as /api/permits/{id}. Updates are validated and stored in PostgreSQL.

3. Delete Operations

Users and administrators can remove records such as reports, permits, or user accounts.

Safety Measures:

Soft deletion is used where appropriate (e.g., is_active flag) to preserve historical data and maintain referential integrity.

Backend Integration:

DELETE requests are processed with validation checks to prevent data inconsistencies.

4. Search Functionality

Users can search for parking lots, reports, and predictions using keywords or time-based queries.

Backend Integration:

Efficient SQL queries with indexed fields are used. Example: searching parking lots by name using case-insensitive matching.

5. Filtering Capabilities

Users can filter parking data based on time, permit type, availability status, and environmental conditions.

Backend Integration:

Filtering is implemented using SQL WHERE clauses and JSONB queries for semi-structured data such as weather conditions.

6. Data Visualization

The system provides visual insights including parking probability heatmaps, usage trends, and predictive analytics.

Data Sources:

- ParkingPrediction

- WeatherSnapshot

- Crowdsourced reports

Frontend Tools:

Libraries such as Chart.js or Google Maps API are used to render graphs and map-based visualizations.

Additional Features

- User authentication and secure login system

- Role-based dashboards (student vs admin)

- Notifications for parking availability alerts

- Pagination for large datasets

- Reporting tools for analytics

B. Interface Layout Design

Main Pages

1. Dashboard

Displays a map of parking lots with color-coded availability indicators and quick statistics.

2. Reports Page

Allows users to submit new reports and view recent submissions.

3. Predictions Page

Shows future parking availability using charts and allows filtering by time and conditions.

4. Admin Panel

Provides tools for managing users, parking lots, and campus events.

5. User Profile

Allows users to view and edit personal information and manage permits.

Navigation Structure

The application uses a top navigation bar for primary access:

Dashboard | Predictions | Reports | Profile

An additional sidebar is available for administrators with links to management and analytics tools.

Page Hierarchy and Flow

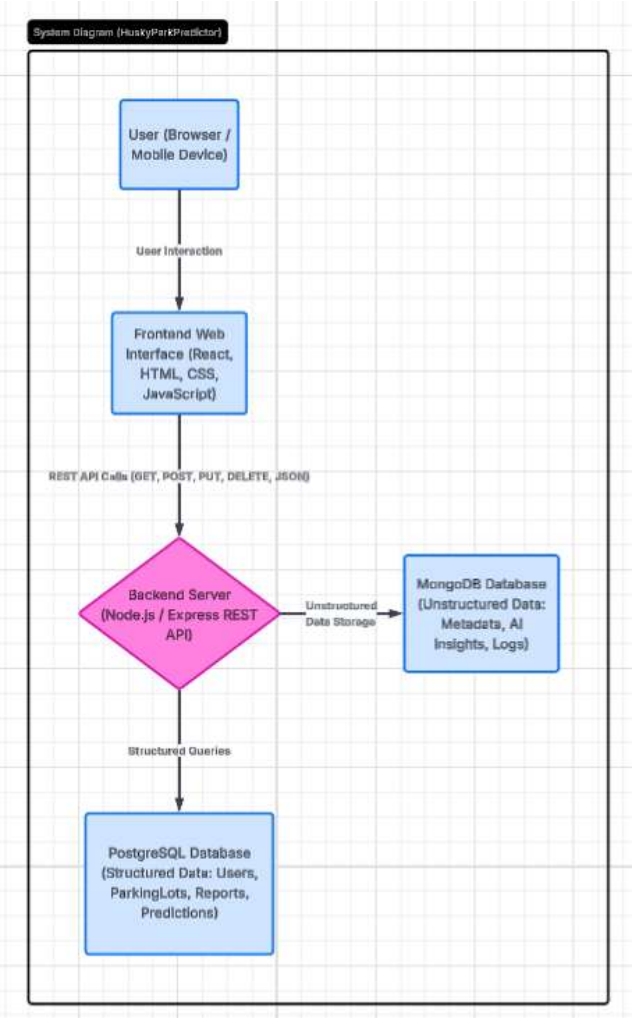
Typical User Flow:

Login → Dashboard → Select Parking Lot → View Prediction → Submit Report

Admin Flow:

Login → Admin Panel → Manage Data → View Analytics

Flowchart:



Usability Considerations

- Consistent layout across all pages
- Map-based interface reduces complexity
- Minimal steps required to complete tasks
- Clear separation between user and admin functionalities

Form Design Examples

Forms follow a simple and structured layout:

- Input fields with labels (e.g., lot selection, availability status)
- Dropdown menus for predefined options
- Submit and reset buttons
- Real-time validation feedback

C. Backend Integration Plan

Frontend-Backend Communication

The system uses a RESTful API architecture. Data is exchanged in JSON format between the frontend and backend.

Example Endpoints:

- GET /api/lots
- POST /api/reports
- GET /api/predictions?lot_id=1

CRUD Operations and Data Flow

Example: Submitting a Parking Report

1. User fills out a form on the frontend
2. A POST request is sent to the backend

3. Backend processes and validates the data

4. Data is stored in PostgreSQL and optionally in MongoDB

5. A success response is returned to the user

Search and Filtering Processing

Search queries are handled through indexed SQL queries for fast performance. Filtering uses both structured queries and JSONB fields for flexible conditions such as weather or time-based filtering.

Data Visualization Integration

The frontend requests prediction and analytics data through API endpoints. The backend aggregates data from multiple tables and returns structured JSON, which is rendered into charts and maps.

Database Integration

PostgreSQL (Relational Database):

- Stores structured data such as users, parking lots, permits, and predictions

MongoDB (NoSQL Database):

- Stores unstructured or semi-structured data such as enriched reports and AI-generated insights

Data Linking Strategy:

Shared identifiers (e.g., report_id) are used to connect records between PostgreSQL and MongoDB.

System Interaction Overview

Frontend → REST API → PostgreSQL → MongoDB

D. Usability & Design Justification

Simplicity and Clarity

The interface prioritizes a clean and intuitive design, using visual elements such as maps and charts to reduce cognitive load.

Accessibility

- High-contrast color schemes for readability
- Mobile-responsive design
- Clearly labeled inputs and accessible navigation

Error Handling and User Feedback

- Immediate validation for form inputs
- Clear success and error messages
- Backend validation ensures data integrity

Performance Considerations

- Indexed database queries for faster search
- Pagination for handling large datasets
- Efficient API design to minimize load times
- Optional caching for frequently accessed predictions

Design Rationale

The map-based dashboard aligns with how users naturally search for parking. Separating structured and unstructured data across PostgreSQL and MongoDB improves scalability and flexibility. Role-based access ensures that both general users and administrators can efficiently perform their tasks.

Conclusion

The HuskyParkPredictor web interface is designed to provide a seamless, efficient, and scalable user experience. By combining strong backend integration with an intuitive frontend design, the system ensures accurate data access, easy navigation, and meaningful insights for users.